

Module 6: Aggregate Functions & Grouping

 **SQL Server Version:** This module has been converted from MySQL to SQL Server (T-SQL) syntax.

Aggregate functions perform calculations on a set of values and return a single result. They are essential for data analysis, reporting, and statistical operations in SQL. When combined with grouping, they enable powerful data summarization and insights.

Understanding Aggregate Functions

Aggregate functions operate on multiple rows of data and consolidate them into a single value. Unlike regular functions that work on individual values, aggregate functions process entire sets of data.

Common Aggregate Functions

SQL provides several built-in aggregate functions for different types of calculations:

Function	Description	Example Use Case
<code>SUM()</code>	Calculates the total of numeric values	Total sales revenue
<code>COUNT()</code>	Counts the number of rows	Number of customers
<code>AVG()</code>	Calculates the average value	Average student score
<code>MIN()</code>	Finds the smallest value	Lowest product price
<code>MAX()</code>	Finds the largest value	Highest employee salary

The SUM Function

The `SUM()` function calculates the total of all numeric values in a specified column. It automatically ignores NULL values.

```
-- Calculate total marks of all students in English
SELECT SUM(english) AS total_english_marks
FROM students;
```

Output:

```
total_english_marks
-----
12450
```

You can use `SUM()` with expressions to calculate derived totals:

```
-- Calculate total marks across all subjects
SELECT SUM(english + maths + science) AS grand_total
FROM students;
```

Output:

```
grand_total
-----
37890
```

The COUNT Function

The `COUNT()` function returns the number of rows that match specified criteria. It has several variations:

COUNT(*)

Counts all rows in the table, including rows with NULL values:

```
-- Count total number of students
SELECT COUNT(*) AS total_students
FROM students;
```

Output:

```
total_students
-----
450
```

COUNT(column_name)

Counts only non-NULL values in a specific column:

```
-- Count students who have marks recorded
SELECT COUNT(maths) AS students_with_maths
FROM students;
```

Output:

```
students_with_maths
-----
448
```

The difference between the two counts indicates there are 2 students with NULL values in the maths column.

The AVG Function

The `AVG()` function calculates the arithmetic mean of numeric values. It automatically excludes NULL values from the calculation.

```
-- Calculate average maths marks
SELECT AVG(maths) AS average_maths
FROM students;
```

Output:

```
average_maths
-----
58.75
```

You can combine `AVG()` with `ROUND()` to control decimal precision:

```
-- Calculate average with 2 decimal places
SELECT ROUND(AVG(science), 2) AS average_science
FROM students;
```

Output:

average_science

61.24

The MIN and MAX Functions

These functions identify the smallest and largest values in a column. They work with numeric, date, and text data types.

```
-- Find lowest and highest marks in English
SELECT MIN(english) AS lowest_marks,
       MAX(english) AS highest_marks
FROM students;
```

Output:

lowest_marks	highest_marks
-----	-----
12	98

With dates, `MIN()` returns the earliest date and `MAX()` returns the most recent:

```
-- Find the oldest and newest employee
SELECT MIN(dob) AS oldest_employee,
       MAX(dob) AS newest_employee
FROM students;
```

Output:

oldest_employee	newest_employee
-----	-----
2005-03-15	2012-11-20

The GROUP BY Clause

The `GROUP BY` clause divides rows into groups based on one or more columns. Aggregate functions then operate on each group separately, producing one result per group.

Basic Grouping

Group students by standard and count how many are in each:

```
-- Count students per standard
SELECT standard, COUNT(*) AS student_count
FROM students
GROUP BY standard
ORDER BY standard;
```

Output:

standard	student_count
-----	-----
1	45
2	48
3	52
4	50
5	47
6	51
7	46
8	49
9	44
10	18

Grouping with Multiple Aggregates

You can use multiple aggregate functions in a single query:

```
-- Calculate statistics for each standard
SELECT standard,
       COUNT(*) AS total_students,
```

```

        ROUND(AVG(maths), 2) AS avg_maths,
        MIN(maths) AS min_maths,
        MAX(maths) AS max_maths
FROM students
GROUP BY standard
ORDER BY standard;

```

Output:

standard	total_students	avg_maths	min_maths	max_maths
-----	-----	-----	-----	-----
1	45	56.20	24	89
2	48	58.35	18	95
3	52	59.40	22	92
4	50	57.85	20	94
5	47	60.12	25	97
6	51	61.45	28	96
7	46	58.90	19	93
8	49	62.30	27	98
9	44	59.75	23	95
10	18	64.80	35	98

Grouping by Multiple Columns

You can group data using multiple columns to create more detailed breakdowns:

```

-- Count students by standard and division
SELECT standard, division, COUNT(*) AS student_count
FROM students
GROUP BY standard, division
ORDER BY standard, division;

```

Output:

standard	division	student_count
-----	-----	-----
1	A	15
1	B	16
1	C	14
2	A	16
2	B	17
2	C	15

3	A	18
3	B	17
3	C	17

Grouping with Calculations

Calculate total marks for each student group:

```
-- Calculate average total marks per standard
SELECT standard,
       ROUND(AVG(english + maths + science), 2) AS avg_total_marks
FROM students
GROUP BY standard
ORDER BY avg_total_marks DESC;
```

Output:

standard	avg_total_marks
10	189.45
8	184.20
6	182.75
5	179.60
3	176.85
9	176.40
7	174.30
4	172.15
2	171.20
1	168.90

The HAVING Clause

The **HAVING** clause filters groups after aggregation. While **WHERE** filters individual rows before grouping, **HAVING** filters the results after grouping.

WHERE vs HAVING

Aspect	WHERE	HAVING
Filter timing	Before grouping	After grouping
Can use with	Column values	Aggregate results

Aspect	WHERE	HAVING
Used with	Any query	GROUP BY queries

Basic HAVING Example

```
-- Find standards with more than 48 students
SELECT standard, COUNT(*) AS student_count
FROM students
GROUP BY standard
HAVING COUNT(*) > 48
ORDER BY student_count DESC;
```

Output:

standard	student_count
3	52
6	51
4	50
8	49

HAVING with Multiple Conditions

```
-- Find standards with average maths marks above 60
SELECT standard,
       ROUND(AVG(maths), 2) AS avg_maths,
       COUNT(*) AS student_count
FROM students
GROUP BY standard
HAVING AVG(maths) > 60
ORDER BY avg_maths DESC;
```

Output:

standard	avg_maths	student_count
10	64.80	18
8	62.30	49
6	61.45	51

Combining WHERE and HAVING

You can use both clauses in the same query. `WHERE` filters rows first, then grouping occurs, and finally `HAVING` filters the groups:

```
-- Find standards from Pune with more than 15 students
SELECT standard, address, COUNT(*) AS student_count
FROM students
WHERE address = 'Pune'
GROUP BY standard, address
HAVING COUNT(*) > 15
ORDER BY student_count DESC;
```

Output:

standard	address	student_count
-----	-----	-----
6	Pune	18
3	Pune	17
8	Pune	17
2	Pune	16

The DISTINCT Keyword

The `DISTINCT` keyword removes duplicate values from query results. It can be used alone or with aggregate functions.

DISTINCT in SELECT

```
-- Find all unique addresses
SELECT DISTINCT address
FROM students
ORDER BY address;
```

Output:

address

Bangalore
Chennai
Delhi

Hyderabad
Mumbai
Pune

DISTINCT with Multiple Columns

```
-- Find unique standard-division combinations
SELECT DISTINCT standard, division
FROM students
ORDER BY standard, division;
```

Output:

standard	division
-----	-----
1	A
1	B
1	C
2	A
2	B
2	C
3	A
3	B
3	C

COUNT with DISTINCT

Count unique values rather than total rows:

```
-- Count how many different cities students come from
SELECT COUNT(DISTINCT address) AS unique_cities
FROM students;
```

Output:

unique_cities

6

Compare this with counting all addresses:

```
-- Compare distinct vs all
SELECT COUNT(address) AS total_records,
       COUNT(DISTINCT address) AS unique_cities
FROM students;
```

Output:

total_records	unique_cities
450	6

Practical Applications

Sales Analysis

```
-- Analyze product sales by category
SELECT p_cat AS category,
       COUNT(*) AS products_sold,
       SUM(price) AS total_revenue,
       ROUND(AVG(price), 2) AS avg_price,
       MIN(price) AS cheapest_item,
       MAX(price) AS most_expensive
FROM products
GROUP BY p_cat
ORDER BY total_revenue DESC;
```

Output:

category	products_sold	total_revenue	avg_price	cheapest_item	most_expensive
Electronics	45	67890.00	1508.67	150.00	45000.00
Foods	38	12450.50	327.65	10.00	2500.00
Stationary	27	3890.75	144.10	12.50	850.00

Student Performance Report

```
-- Create a comprehensive performance report
SELECT standard,
       COUNT(*) AS total_students,
       ROUND(AVG(english), 2) AS avg_english,
       ROUND(AVG(maths), 2) AS avg_maths,
       ROUND(AVG(science), 2) AS avg_science,
       ROUND(AVG(english + maths + science), 2) AS avg_total
FROM students
GROUP BY standard
HAVING AVG(english + maths + science) > 170
ORDER BY avg_total DESC;
```

Output:

standard	total_students	avg_english	avg_maths	avg_science	avg_total
10	18	62.40	64.80	62.25	189.45
8	49	60.50	62.30	61.40	184.20
6	51	59.85	61.45	61.45	182.75
5	47	58.90	60.12	60.58	179.60
3	52	57.80	59.40	59.65	176.85
9	44	58.25	59.75	58.40	176.40
7	46	57.60	58.90	57.80	174.30
4	50	56.95	57.85	57.35	172.15
2	48	56.20	58.35	56.65	171.20

Employee Salary Statistics

```
-- Analyze salary distribution by position
SELECT Position,
       COUNT(*) AS employee_count,
       MIN(Salary) AS min_salary,
       MAX(Salary) AS max_salary,
       ROUND(AVG(Salary), 2) AS avg_salary,
       SUM(Salary) AS total_payroll
FROM EMPLOYEES
GROUP BY Position
ORDER BY avg_salary DESC;
```

Output:

Position	employee_count	min_salary	max_salary	avg_salary	
total_payroll					
-----	-----	-----	-----	-----	

Store Manager	1	35000.00	35000.00	35000.00	35000.00
Inventory Clerk	1	22000.00	22000.00	22000.00	22000.00
Sales Associate	1	20000.00	20000.00	20000.00	20000.00
Cashier	1	18000.00	18000.00	18000.00	18000.00

Advanced Grouping Techniques

Nested Aggregations

While you cannot directly nest aggregate functions, you can use subqueries to achieve similar results:

```
-- Find the standard with the highest average marks
SELECT standard, avg_marks
FROM (
    SELECT standard,
           ROUND(AVG(english + maths + science), 2) AS avg_marks
    FROM students
    GROUP BY standard
) AS standard_averages
WHERE avg_marks = (
    SELECT MAX(avg_marks)
    FROM (
        SELECT ROUND(AVG(english + maths + science), 2) AS avg_marks
        FROM students
        GROUP BY standard
    ) AS max_avg
);
```

Output:

standard	avg_marks
-----	-----
10	189.45

Grouping with CASE Statements

Create custom groupings using conditional logic:

```
-- Categorize students by performance level
SELECT
    CASE
        WHEN maths >= 75 THEN 'Excellent'
        WHEN maths >= 60 THEN 'Good'
        WHEN maths >= 35 THEN 'Average'
        ELSE 'Needs Improvement'
    END AS performance_category,
    COUNT(*) AS student_count,
    ROUND(AVG(maths), 2) AS avg_marks
FROM students
GROUP BY
    CASE
        WHEN maths >= 75 THEN 'Excellent'
        WHEN maths >= 60 THEN 'Good'
        WHEN maths >= 35 THEN 'Average'
        ELSE 'Needs Improvement'
    END
ORDER BY avg_marks DESC;
```

Output:

performance_category	student_count	avg_marks
-----	-----	-----
Excellent	92	82.45
Good	168	67.30
Average	156	48.75
Needs Improvement	34	22.80

Common Mistakes and Best Practices

Invalid: Selecting Non-Grouped Columns

When using `GROUP BY`, you can only select columns that are either in the `GROUP BY` clause or used with aggregate functions:

```
-- This will cause an error
SELECT standard, division, COUNT(*)
FROM students
GROUP BY standard;
-- Error: division must be in GROUP BY or use aggregate function
```

```
-- Correct version
SELECT standard, division, COUNT(*)
FROM students
GROUP BY standard, division;
```

NULL Handling in Aggregates

Aggregate functions automatically ignore NULL values, which affects the results:

```
-- Understanding NULL behavior
SELECT
    COUNT(*) AS total_rows,
    COUNT(marks) AS non_null_marks,
    AVG(marks) AS average_marks,
    SUM(marks) AS total_marks
FROM students;
```

Output:

total_rows	non_null_marks	average_marks	total_marks
450	448	58.75	26320.00

The average is calculated only from the 448 non-NULL values, not all 450 rows.

Using Aliases

Always use meaningful aliases for aggregate results to improve readability:

```
-- Good practice: clear, descriptive aliases
SELECT standard,
    COUNT(*) AS total_students,
    ROUND(AVG(maths), 2) AS average_mathematics_score,
    MAX(maths) AS highest_score,
    MIN(maths) AS lowest_score
FROM students
GROUP BY standard;
```

Performance Considerations

Grouping and aggregation can be resource-intensive operations. Consider these optimization strategies:

- Use indexes on columns frequently used in `GROUP BY` clauses
- Filter data with `WHERE` before grouping to reduce the dataset size
- Limit the number of groups when possible to improve query performance
- Use `HAVING` only when you need to filter on aggregated results

```
-- Efficient query: filter before grouping
SELECT standard, AVG(maths) AS avg_maths
FROM students
WHERE address = 'Pune' -- Filters rows first
GROUP BY standard
HAVING AVG(maths) > 60; -- Then filters groups
```

Summary

Aggregate functions and grouping are fundamental tools for data analysis in SQL. Key takeaways:

- `SUM()`, `COUNT()`, `AVG()`, `MIN()`, and `MAX()` perform calculations on sets of data
- `GROUP BY` divides data into groups for separate aggregation
- `HAVING` filters groups based on aggregate conditions
- `DISTINCT` eliminates duplicate values from results
- Aggregate functions ignore NULL values automatically
- Combine multiple aggregate functions for comprehensive analysis
- Use `WHERE` to filter rows before grouping and `HAVING` to filter groups after aggregation

Mastering these concepts enables you to transform raw data into meaningful insights, create reports, and perform statistical analysis efficiently. Practice combining these functions with joins and subqueries to handle increasingly complex analytical requirements.